

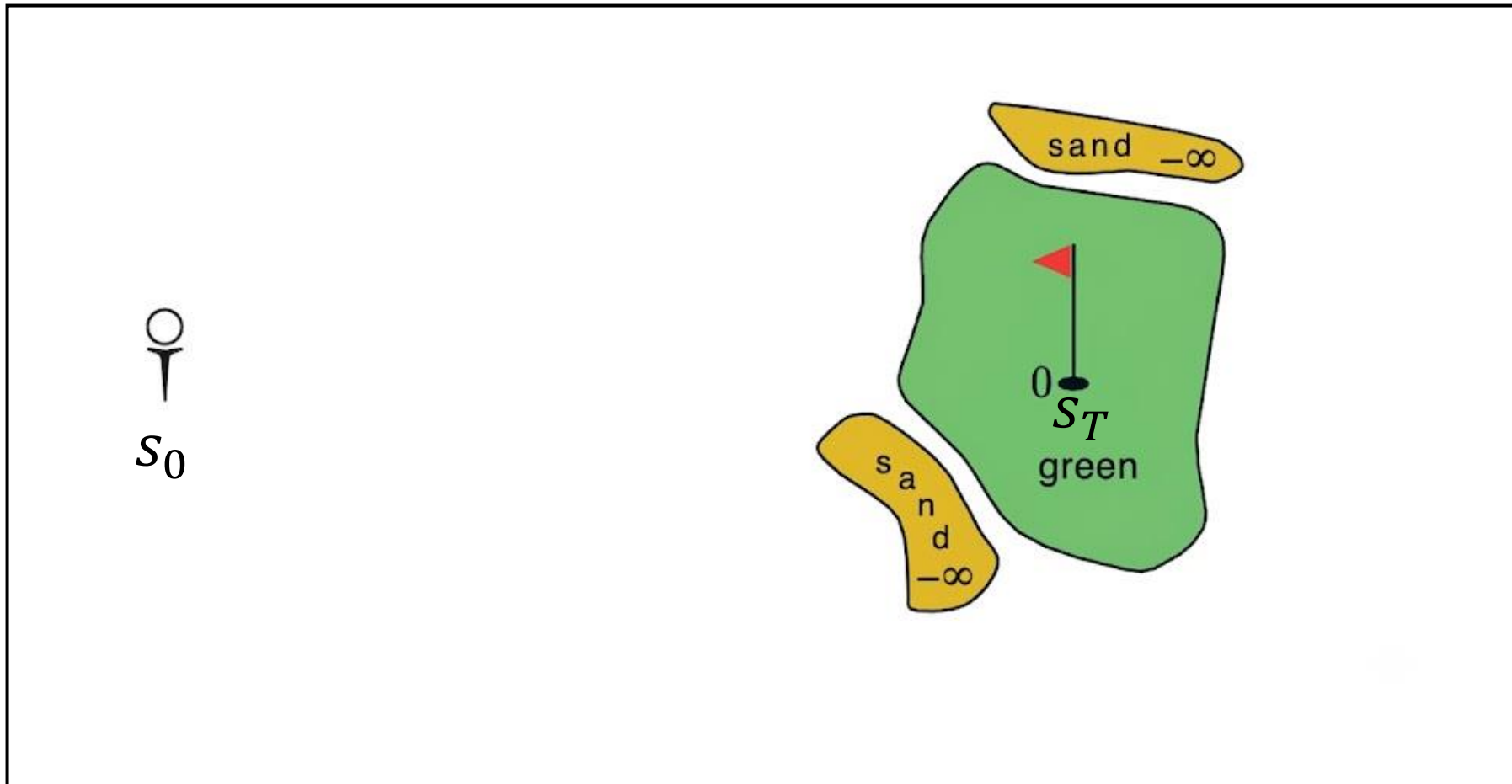
Reinforcement Learning: Planning by Dynamic Programming (ESE 3990)

Instructor:
Antonio Loquercio

Fall 2026
University of Pennsylvania

Example: Playing Golf

$$s \in S$$



$$a \in A$$



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Playing Golf

- Finding $V^*(s)$. Algorithm intuition:
 - Start from the states *you know* you can reach the hole with a single action (either putt or driver). Let's call this set $S^1 \subset S$.
What is $V^*(s), \forall s \in S^1$?
 - Now go to the states you know you can reach a state in S^1 with a single action. Let's call this set $S^2 \subset S$.
What is $V^*(s), \forall s \in S^2$?
 - Continue up to when you have covered *all states* in S .
Return $V^*(s), \forall s \in S$.



Driver



Putt

$$r(s, a) = -1$$

$$a \in A$$

Dynamic Programming: Playing Golf

- Algorithm intuition:
 - Step 1: First find $V^*(s)$ using recursive search.
 - Step 2: $\forall s \in S$, find $\pi^*(s)$ as:
$$\pi^*(s) = \arg \max_a r(s, a) + \gamma \mathbb{E}_{\mathbb{P}(s_{t+1}|s)} [V^*(s_{t+1}) | s, a]$$
- Recursive search is only possible when, given a state, I can predict predecessor states.
 - Formally, this requires inverting the transition matrix:
$$s_t = (P^\pi)^{-1} s_t$$
- This won't work if:
 - The MDP is loopy (P is not invertible).
 - The number of states is very large (>millions)



Driver



Putt

$$r(s, a) = -1$$

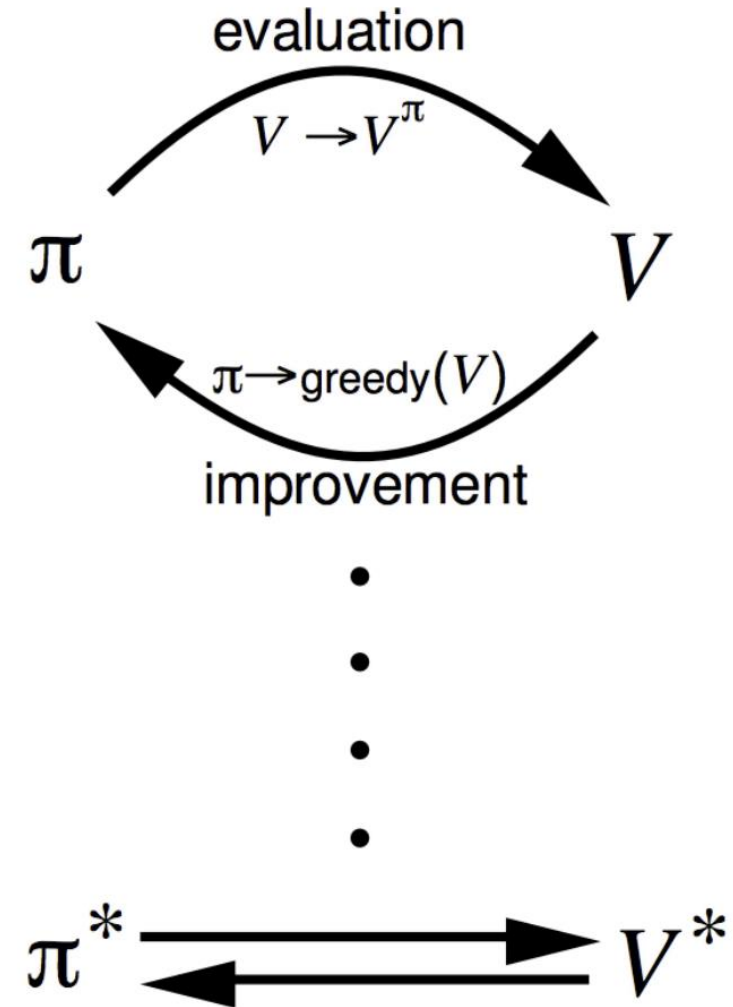
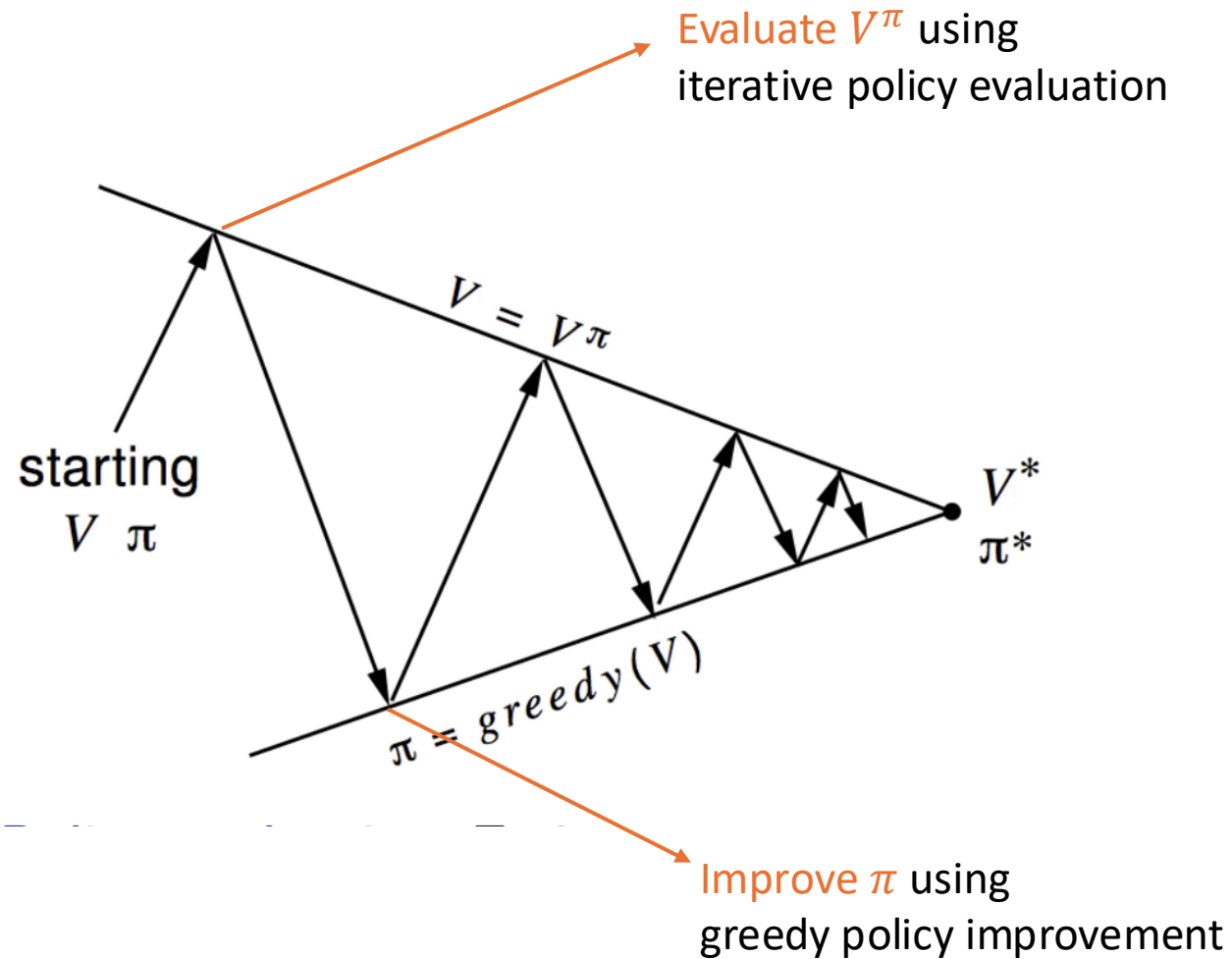
Iterative Policy Evaluation: V^π w/o Matrix Inversion

- Problem: Given a policy π , find its value function V_π .
- Algorithm (*synchronous* backups):
 - Initialize $V_0 = 0$
 - At each iteration $k + 1$
 - $\forall s \in S$, update $V_{k+1}(s)$ from $V_k(s')$, where s' is a successor state of s


$$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s') \right)$$

- **Idea:** Iterative application of Bellman expectation backup.
$$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^\pi$$
- The intermediate V_k are not necessarily value function of any policy.
- We can prove asymptotic convergence to V^π

Policy Iteration



Value Iteration:

The idea of **Value Iteration** is applying the Bellman *Optimality* Equation to V_k :

$$V_{k+1}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s')$$

- The only case where LHS is equal to RHS for all states is for V^*
- No explicit policy
- Computationally much cheaper than Policy Iteration and still converges to the optimal V^*

Value Iteration: Finding V^* Directly

- Init $V_0(s) = 0, \forall s \in S$



- At each iteration $k + 1$

- $\forall s \in S$, update $V_{k+1}(s)$ from $V_k(s')$:

$$V_{k+1}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_k(s')$$

- Idea: $V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V^*$
- This converges to V^* , even for loopy, stochastic MDP!
- Can be proven using Bellman Optimality Principle.

Example: Shortest Path

g			

Problem

Summary of Synchronous DP Algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- The algorithms are based on state-value function $V^\pi(s)$ or $V^*(s)$
 - The complexity is $O(mn^2)$ per iteration, for m actions and n states.
- Could also apply the same idea to Q functions $Q_\pi(s, a)$ or $Q^*(s, a)$
 - Complexity is $O(m^2n^2)$

Asynchronous Dynamic Programming

- Up to now, we have seen synchronous methods that update all states in parallel.
- *Asynchronous DP* backs up states individually, in any order
 - For each selected state, apply the appropriate backup
 - Can significantly reduce computation
 - *Guaranteed* to converge if all states continue to be visited.

Asynchronous Dynamic Programming

- Three simple ideas for asynchronous dynamic programming:
 - *In-place* dynamic programming
 - *Prioritized Sweeping*
 - *Real-time* dynamic programming

In-Place Dynamic Programming

- Synchronous value iteration stores two copies of value function

$$\forall s \in S, V_{new}(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V_{old}(s')$$
$$V_{old} \leftarrow V_{new}$$

- In-place value iteration only stores one copy of value function

$$\forall s \in S, V(s) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V(s')$$

Prioritized Sweeping

- *Intuition:* Backup the states with the largest remaining Bellman errors.

- Formally, use magnitude of Bellman error to guide state selection:

$$\left| \max_a \left(r(s, a) + \gamma \sum_{s' \in S} P_{ss'}^a V(s') \right) - V(s) \right|$$

- After each backup, update the Bellman error of affected states
 - Can be implemented efficiently by maintaining a priority queue

Real-time Dynamic Programming

- *Idea:*

- update only states that are relevant to the agent.

- *Implementation:*

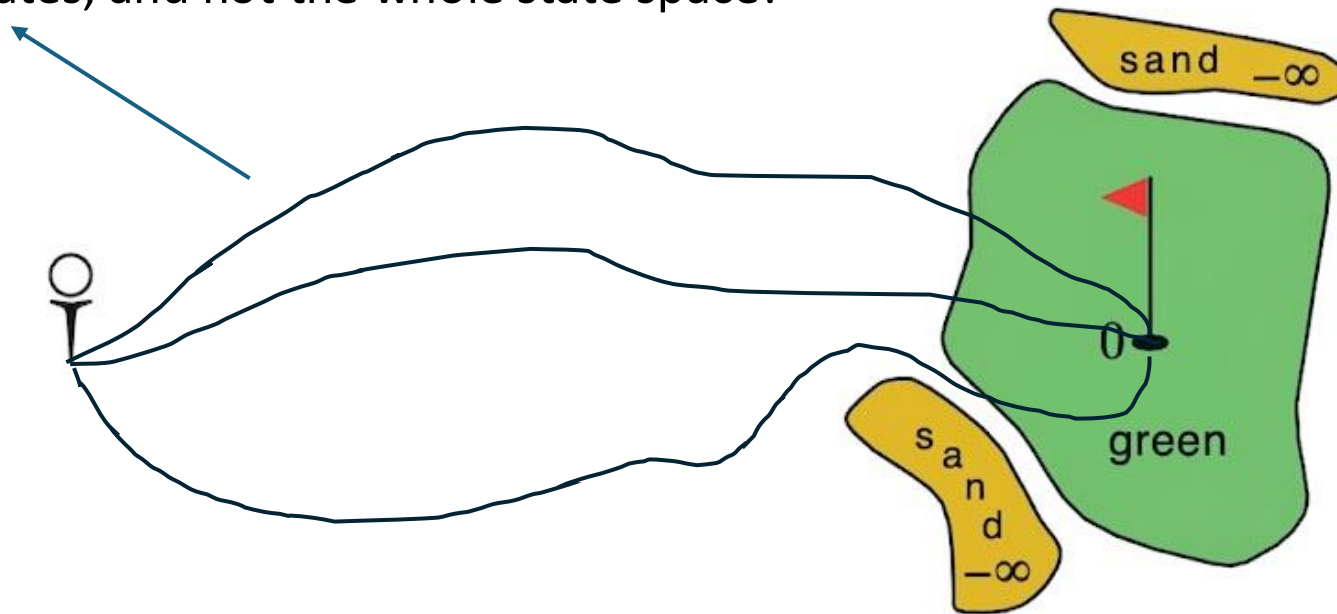
- Use the agent's experience to guide the selection of state.
- Formally, after each time-step s_t, a_t, r_{t+1}
- backup *only* the state s_t

$$V(s_t) = \max_{a \in A} r(s, a) + \gamma \sum_{s' \in S} P_{s_t s'}^a V(s')$$

Real-time Dynamic Programming

- *Idea*: update only states that are relevant to the agent.

Only update these states, and not the whole state space!



Dynamic Programming vs Sampling

Full-Width Backups

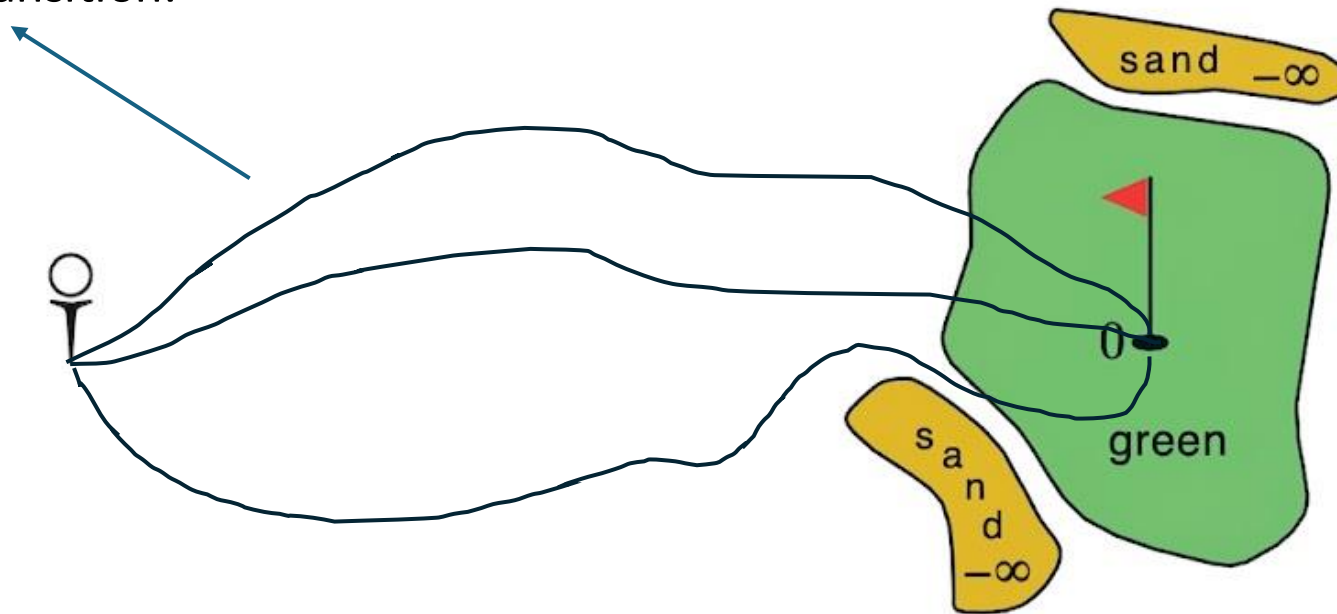
- DP uses *full-width* backups:
 - For each backup (sync or async)
 - **Every** successor state and action is considered ($P_{ss'}^a, \forall a \in A, \forall s' \in S$)
 - We can do this because we assume *full knowledge of the MDP*
- DP is effective for medium-sized tabular problems (million of states)
 - For larger problems, even if we know the MDP, every backup can be too expensive.
 - This is known as Bellman's *curse of dimensionality*
 - Note: number of states $n = |S|$ grows exponentially with the number of state variables.

Sample Backups

- Next, we will consider *sample backups*.
 - Updates use sample rewards and *sample transitions* $\langle S, A, R, S' \rangle$ collected by a policy
 - Requires no knowledge of the transition dynamics $P_{ss'}^a, \forall a \in A, \forall s' \in S$.
 - We call this *model-free*.
- *Advantages:*
 - “Breaks” the curse of dimensionality through sampling
 - Cost of backup is constant, independent of $n = |S|$

Sample Backups

Only update these states, using the successor state s' observed in the transition.



Technical Questions [No Exam]

- How do we know that value iteration converges to V_* ?
- Or that iterative policy evaluation converges to V_π ?
 - And therefore, that policy iteration converges to V_* ?
- Is the solution unique?
- How fast do these algorithms converge (if they do)?
- These questions are resolved by the *contraction mapping theorem*

Value Function Space

- Consider the vector space $\mathcal{V}$ over value functions
 - There are $|S|$ dimensions
 - $\forall s \in S, V(s)$ can have any value in $[-\infty, +\infty]$
 - Each point in this space fully specifies a value function $V(s)$
- We will show that the Bellman backup brings points in this space closer
 - For example, after each backup, the distance between $V_{old}(s)$ and $V_{new}(s)$ decreases.
 - Therefore, the backups *must* converge on a unique solution.

Bellman Expectation Backup is a Contraction

- See blackboard derivation

Contraction Mapping Theorem

Theorem (Contraction Mapping):

- For any metric space $\mathcal{V}$ that is complete (i.e., closed) under and operator $T(v)$, where T is a γ -contraction:
 - T converges to a unique fixed point
 - At a linear convergence rate of γ

Convergence Of Iter. Policy Evaluation and Policy Iteration

- The Bellman expectation operator T^π has a unique fixed point
- V_π is a fixed point of T^π (by Bellman expectation equation)
- By contraction mapping theorem:
 - Iterative policy evaluation converges on V_π
 - Policy Iteration converges on V_*

Bellman Optimality Backup is a Contraction

- Define the Bellman optimality backup operator T^* ,

$$T^*(v) = \max_{a \in A} R^a + \gamma P^a v$$

- This operator is a γ -contraction
 - It makes value functions closer by at least γ (similar to prior proof)

$$\|T^*(u) - T^*(v)\|_\infty \leq \gamma \|u - v\|_\infty$$

Converge of Value Iteration

- The Bellman optimality operator T^* has a unique fixed point
- V_* is a fixed point of T^* (by Bellman optimality equation)
- By contraction mapping theorem, value iteration converges to V_*
- Summary: Almost all algorithms presented today converge to the optimal solution!
 - It won't happen again during this class 😞